

NATIONAL UNIVERSITY OF SCIENCES AND TECHNOLOGY

SCHOOL OF CIVIL AND ENVIRONMENTAL ENGINEERING (SCEE)

INSTITUTE OF GEOGRAPHICAL INFORMATION SYSTEMS (IGIS)

Instructor: Ms. Alvina Anjum

Course: CS 471 Machine Learning (CS471 F'25 BEGI2k22)

COMPLEX ENGINEERING PROBLEM

Rainfall Prediction

Group Members:

Faatima Aamir 404214

Hijab Zahra 421132

Noor-ul-Haya 425722

1. Abstract

This proposal proposes a solution to the problem of rainfall pattern predictions in a situation where climate variability is rising by developing a machine learning model that is coupled with a Geographic Information System (GIS). The problem that this proposal proposes to solve is developing a predictive tool that is capable of predicting rainfall amount or probability for a certain region by combining spatial variables (which are obtained from landcover, DEM, and position), as well as temporal variables (such as past rainfall, temperatures, and humidity). This proposal proposes a solution that entails a rigorous process of preprocessing the different datasets, which is performed on an AWS EC2 instance, where Python libraries such as rasterio & pandas are used to extract, merge, and prepare the different datasets, which include daily CHIRPS precipitation totals that are aggregated to monthly totals. Following that, different models such as Linear Regression, Random Forest, & XGBoost models are going to be created on the AWS instance. The system that is going to be created is capable of providing a precise, optimized solution for a problem within the environment, providing a graphical solution for predicting precipitation, as well as providing a solution regarding how spatial attributes, as well as temporal attributes, affect rainfall conditions within a certain region.

2. Introduction

2.1 Project Objective and Context

Accurate rainfall prediction has become a crucial challenge with increasing climatic variability and the non-linear dynamics that are characteristic of the weather system; therefore, this requires a new approach.

It uses ML integrated with the data coming out of GIS as an enabler for constructing a data-driven rain prediction system.

Problem Statement:

Design an ML model that can predict amounts of rainfall by considering integrated spatial features like DEM and Landcover, along with temporal variables such as past rainfall and temperature. The proposed system will align heterogeneous datasets of the environment and will be able to produce predictive maps for visualization and further analysis.

The project is composed of the following four core objectives: Data Alignment & Feature Engineering: Gathering and preprocessed collection of all spatial (CHIRPS, Landcover, DEM) and temporal datasets, standardizing them to a uniform resolution (EPSG:4326, 0.05 degrees) and aggregate them into monthly totals for ML readiness. Model Development: Compare multiple predictive models (Linear Regression, Random Forest, XGBoost) to determine the most robust algorithm on the spatial-temporal data. Predictive Analysis: To quantify the contribution and influence of both spatial and temporal factors on rainfall patterns. Evaluation & Visualization: Rigorous model performance evaluation using metrics (Rsquared, RMSE, MAE) and development of practical GIS-based rainfall distribution maps.

2.2 Data Sources

FEATURE CATEGORY	DATA SOURCE (ACRONYM)	LIKELY ORIGIN/REPOSITORY	NOTES
RAINFALL (TEMPORAL)	CHIRPS	UCSB CHC (Climate Hazards Group) or accessed via Google Earth Engine (GEE).	This is derived from satellite (IR) and ground station data. You used the daily data.

TEMPERATURE/HUMIDITY (TEMPORAL)	NASA POWER	NASA Prediction of Worldwide Energy Resources API/Platform.	Provides long-term climatology data derived from NASA satellites and models. You would typically access this via an API or portal.
VEGETATION INDEX (SPATIAL/TEMPORAL)	NDVI	Derived from Landsat or MODIS satellite imagery, often accessed and processed via Google Earth Engine (GEE).	NDVI is a common metric. Specify the source satellite if possible (e.g., MODIS/Terra + Aqua or Landsat 8/9).
LAND COVER (SPATIAL)	ESA WorldCover	European Space Agency official repository, often accessed via Google Earth Engine (GEE) or ESA's Copernicus portal.	This is a high-resolution, static global land cover map for a specific year (e.g., 2020 or 2021).
ELEVATION (STATIC SPATIAL)	DEM	USGS (U.S. Geological Survey) or NASA SRTM (Shuttle Radar Topography Mission) data, often accessed through an open repository or GEE.	Specify the resolution (e.g., 30-meter resolution) if known.

3. Methodology

3.1 Study Design Overview

This study aims to predict daily rainfall intensity using machine learning and deep learning approaches by integrating satellite-based rainfall observations with meteorological and vegetation indicators. Due to the highly zero-inflated and temporally dependent nature of rainfall data, a structured modeling pipeline was designed that ensures fair comparison across models while remaining computationally feasible.

Three models were implemented and compared:

- XGBoost (gradient-boosted decision trees)
- LightGBM (histogram-based gradient boosting)
- Long Short-Term Memory (LSTM) neural network

All models were trained and evaluated using identical input data, features, and temporal splits to ensure methodological consistency.

3.2 Data Sources and Preprocessing

3.2.1 Datasets

The following datasets were used:

- **CHIRPS**: Daily satellite-derived rainfall estimates (mm)
- **NASA POWER**: Meteorological variables including air temperature (T2M), relative humidity (RH2M), wind speed (WS2M), and solar radiation
- **NDVI**: Vegetation index representing land surface conditions

The datasets were spatially aligned using latitude and longitude coordinates and temporally aligned using daily timestamps.
To ensure computational feasibility, **only data from the year 2025** was used in model training and evaluation.

3.2.2 Handling Spatial and Temporal Alignment

- All datasets were standardized to common coordinate systems.
- Nearest-neighbor spatial matching was applied where exact coordinate alignment was unavailable.
- Temporal consistency was enforced by filtering records to daily resolution

3.2.3 Feature Engineering

To capture rainfall persistence and short-term temporal dependency, **lag-based features** were created:

- Rainfall at **1-day, 3-day, and 7-day lags**

These lag features enable tree-based models to learn temporal patterns and provide meaningful historical context for the LSTM model.
The final feature set included:

- Meteorological variables (T2M, RH2M, WS2M, solar radiation)
- NDVI
- Rainfall lag features

3.3 Two-Stage Modeling Framework

Rainfall data is characterized by a high frequency of zero values (no rainfall days). To address this **zero inflation**, a **two-stage modeling framework** was adopted.

Stage 1: Rain Occurrence Classification

A binary classification task was first performed to predict whether rainfall would occur on a given day:

$$\text{Rain Occurrence} = \begin{cases} 1, & \text{if rainfall} > 0 \\ 0, & \text{if rainfall} = 0 \end{cases}$$

This stage reduces bias in rainfall magnitude prediction by separating dry and wet days.

Stage 2: Rainfall Intensity Regression

Regression models were trained **only on days classified as rainy** to predict rainfall magnitude (mm).
This approach improves model stability and reduces error introduced by excessive zero values.

3.4 Model Implementation

3.4.1 XGBoost

XGBoost was implemented using a gradient boosting framework with histogram-based tree construction.
Key characteristics:

- Handles nonlinear relationships efficiently
- Robust to multicollinearity
- Suitable for structured tabular data

Incremental and batch training strategies were applied depending on computational constraints.

3.4.2 LightGBM

LightGBM was used as a computationally efficient alternative to XGBoost, leveraging:

- Leaf-wise tree growth
- Histogram-based feature binning
- Reduced memory consumption

LightGBM enabled faster convergence while maintaining competitive predictive performance.

3.4.3 LSTM

The LSTM model was designed to explicitly capture **temporal dependencies** in rainfall sequences.

- Input sequences were constructed using fixed-length sliding windows
- Lag features and meteorological variables were included as multivariate inputs
- Mean Squared Error was used as the loss function

Due to computational limitations, the LSTM model was trained on a reduced spatial subset and limited epochs.

3.5 Training Strategy and Evaluation

3.5.1 Temporal Train-Test Split

To prevent data leakage, a **time-based split** was applied:

- Early months of 2025 used for training
- Later months used for testing

This setup reflects real-world forecasting conditions.

3.5.2 Evaluation Metrics

Model performance was evaluated using the following metrics:

- **Root Mean Squared Error (RMSE)**
- **Mean Absolute Error (MAE)**
- **Coefficient of Determination (R^2)**

RMSE was used as the primary metric due to its sensitivity to large rainfall prediction errors.

3.6 Visualization and Interpretation

Model outputs were evaluated using:

- Actual vs. predicted rainfall time-series plots at selected locations
- Loss curves for LSTM training
- Comparative performance tables across models

These visualizations support both quantitative and qualitative assessment of model performance.

3.7 Summary of Methodological Strengths

- Explicit handling of zero-inflated rainfall data
- Fair comparison across models using identical data and features
- Inclusion of temporal lag features
- Computationally efficient design suitable for limited resources

4. Code Used for each Step

4.1 XGBoost

```
import numpy as np
import xgboost as xgb # Using XGBoost
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score, mean_squared_log_error
from scipy.spatial import cKDTree
import matplotlib.pyplot as plt

# --- CONFIGURATION (Based on your best LGBM results) ---
# We use the parameters from your winning LightGBM model as a strong starting point for XGBoost
best_xgb_params = {
    'max_depth': 8,
    'gamma': 0.1, # Equivalent to min_split_loss or min_child_weight
    'lambda': 1,  # L2 regularization (similar to reg_lambda)
}

# --- 2. Data Splitting and Training (XGBoost with BEST OBJECTIVE) ---

print("⏳ Training XGBoost model with objective='reg:squarederror' (L2/RMSE) ...")
# XGBoost's standard objective for L2 loss minimization (RMSE)
xgb_model = xgb.XGBRegressor(
    objective='reg:squarederror', # <--- BEST PERFORMING OBJECTIVE (L2 Loss)
    n_estimators=500,
    learning_rate=0.05,
    subsample=0.7,
    colsample_bytree=0.7,
    random_state=42,
    n_jobs=-1,
    tree_method='hist',
    **best_xgb_params
)
xgb_model.fit(X_train, y_train)
preds = xgb_model.predict(X_test)
```

```python
# --- 3. Evaluation and Visualization ---

# Reconstruct test dataframe (assuming X_test_df and chirps are available)
test_df = chirps.loc[X_test_df.index].copy()
test_df["predicted_rainfall"] = preds

# CALCULATE SCORES
rmse = np.sqrt(mean_squared_error(y_test, preds))
mae = mean_absolute_error(y_test, preds)
r2 = r2_score(y_test, preds)
msle = mean_squared_log_error(np.clip(y_test, 1e-9, None), np.clip(preds, 1e-9, None)) # Ensure positive for log

print("\n========================================")
print("☔ Regression Model Evaluation Scores (XGBoost - L2/RMSE Objective)")
print("========================================")
print(f"1. RMSE (Root Mean Squared Error):     {rmse:.6f}")
print(f"2. MAE (Mean Absolute Error):          {mae:.6f}")
print(f"3. R² Score (Coefficient of Det.):     {r2:.6f}")
print(f"4. MSLE (Mean Squared Log Error):      {msle:.6f}")
print("========================================")


# PLOT TIME SERIES
sample_point = test_df[["lat","lon"]].drop_duplicates().sample(1, random_state=42)
lat, lon = sample_point.iloc[0]
sample_ts = test_df[
    (test_df["lat"] == lat) & (test_df["lon"] == lon)
].sort_values("date")

plt.figure(figsize=(10,4))
plt.plot(sample_ts["date"], sample_ts["rainfall_mm"], label="Actual")
plt.plot(sample_ts["date"], sample_ts["predicted_rainfall"], label="Predicted")
plt.xlabel("Date")
```

```python
plt.xlabel("Date")
plt.ylabel("Rainfall (scaled)")
plt.title(f"Rainfall Prediction Time Series (2025) - XGBoost (L2/RMSE)\nLat={lat:.2f}, Lon={lon:.2f}")
plt.legend()
plt.tight_layout()
plt.show()


# DISTRIBUTION PLOT
plt.figure(figsize=(8, 4))
bins = 50
plt.hist(test_df["rainfall_mm"], bins=bins, density=True, alpha=0.6, label="Actual Rainfall")
plt.hist(test_df["predicted_rainfall"], bins=bins, density=True, alpha=0.6, label="Predicted Rainfall")
plt.xlabel("Rainfall (scaled)")
plt.ylabel("Density")
plt.title("Distribution of Actual vs. Predicted Rainfall (XGBoost - L2/RMSE)")
plt.legend()
plt.tight_layout()
plt.show()
```

```
⏳ Training XGBoost model with objective='reg:squarederror' (L2/RMSE) ...

========================================
☔ Regression Model Evaluation Scores (XGBoost - L2/RMSE Objective)
========================================
1. RMSE (Root Mean Squared Error):     0.051598
2. MAE (Mean Absolute Error):          0.036647
3. R² Score (Coefficient of Det.):     0.520453
4. MSLE (Mean Squared Log Error):      0.002120
========================================
```

```python
import pandas as pd
import numpy as np
import xgboost as xgb
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score, mean_squared_log_error

# --- CONFIGURATION (Based on your best LGBM results) ---
best_xgb_params = {
    'max_depth': 8,
    'gamma': 0.1,
    'lambda': 1,
}

# NOTE: X_train, X_test, y_train, y_test, chirps, and X_test_df are assumed to be defined.

# --- 2. Data Splitting and Training (XGBoost with MAE OBJECTIVE) ---

print("⏳ Training XGBoost model with objective='reg:absoluteerror' (MAE)...")
# XGBoost's objective for L1 loss minimization (MAE)
xgb_model_mae = xgb.XGBRegressor(
    objective='reg:absoluteerror', # <--- CRITICAL CHANGE to MAE Objective
    n_estimators=500,
    learning_rate=0.05,
    subsample=0.7,
    colsample_bytree=0.7,
    random_state=42,
    n_jobs=-1,
    tree_method='hist',
    **best_xgb_params
)
xgb_model_mae.fit(X_train, y_train)
preds_mae = xgb_model_mae.predict(X_test)
```

```python
# --- 3. Evaluation and Visualization ---

# Reconstruct test dataframe
test_df = chirps.loc[X_test_df.index].copy()
test_df["predicted_rainfall"] = preds_mae

# CALCULATE SCORES
rmse = np.sqrt(mean_squared_error(y_test, preds_mae))
mae = mean_absolute_error(y_test, preds_mae)
r2 = r2_score(y_test, preds_mae)
msle = mean_squared_log_error(np.clip(y_test, 1e-9, None), np.clip(preds_mae, 1e-9, None))

print("\n==========================================")
print("☔ Regression Model Evaluation Scores (XGBoost - MAE Objective)")
print("==========================================")
print(f"1. RMSE (Root Mean Squared Error):     {rmse:.6f}")
print(f"2. MAE (Mean Absolute Error):          {mae:.6f}")
print(f"3. R² Score (Coefficient of Det.):     {r2:.6f}")
print(f"4. MSLE (Mean Squared Log Error):      {msle:.6f}")
print("==========================================")


# PLOT TIME SERIES
sample_point = test_df[["lat","lon"]].drop_duplicates().sample(1, random_state=42)
lat, lon = sample_point.iloc[0]
sample_ts = test_df[
    (test_df["lat"] == lat) & (test_df["lon"] == lon)
].sort_values("date")
```

```
plt.figure(figsize=(10,4))
plt.plot(sample_ts["date"], sample_ts["rainfall_mm"], label="Actual")
plt.plot(sample_ts["date"], sample_ts["predicted_rainfall"], label="Predicted")
plt.xlabel("Date")
plt.ylabel("Rainfall (scaled)")
plt.title(f"Rainfall Prediction Time Series (2025) - XGBoost (MAE)\nLat={lat:.2f}, Lon={lon:.2f}")
plt.legend()
plt.tight_layout()
plt.show()


# DISTRIBUTION PLOT
plt.figure(figsize=(8, 4))
bins = 50
plt.hist(test_df["rainfall_mm"], bins=bins, density=True, alpha=0.6, label="Actual Rainfall")
plt.hist(test_df["predicted_rainfall"], bins=bins, density=True, alpha=0.6, label="Predicted Rainfall")
plt.xlabel("Rainfall (scaled)")
plt.ylabel("Density")
plt.title("Distribution of Actual vs. Predicted Rainfall (XGBoost - MAE)")
plt.legend()
plt.tight_layout()
plt.show()
```

```
⏳ Training XGBoost model with objective='reg:absoluteerror' (MAE)...

========================================
☔ Regression Model Evaluation Scores (XGBoost - MAE Objective)
========================================
1. RMSE (Root Mean Squared Error):     0.051494
2. MAE (Mean Absolute Error):          0.032619
3. R² Score (Coefficient of Det.):     0.522385
4. MSLE (Mean Squared Log Error):      0.002087
========================================
```

```python
import pandas as pd
import numpy as np
import xgboost as xgb
import matplotlib.pyplot as plt

# Assuming 'xgb_model_mae' is the best XGBoost model trained with MAE objective.
# Assuming 'X_train' (or FEATURES list) holds the feature names.

print("\n--- 1. Generating XGBoost Feature Importance Plot ---")

# Extract feature importances
feature_importances = pd.Series(xgb_model_mae.feature_importances_, index=X_train_df.columns)
# Use the full X_train_df to get feature names if X_train is a numpy array
# If X_train is a numpy array, you would use the FEATURES list for indexing.

# Plotting the top 15 features
plt.figure(figsize=(10, 6))
feature_importances.nlargest(15).sort_values(ascending=True).plot(kind='barh')
plt.title("Top 15 Feature Importances (XGBoost)")
plt.xlabel("Feature Importance Score (Gain)")
plt.tight_layout()
plt.show()


print("\n--- 2. Generating Time Series Comparison Plot (Focus Zoom) ---")

# Reconstruct test dataframe (from previous step)
test_df = chirps.loc[X_test_df.index].copy()
preds_mae = xgb_model_mae.predict(X_test)
test_df["predicted_rainfall"] = preds_mae
```

```python
print("\n--- 2. Generating Time Series Comparison Plot (Focus Zoom) ---")

# Reconstruct test dataframe (from previous step)
test_df = chirps.loc[X_test_df.index].copy()
preds_mae = xgb_model_mae.predict(X_test)
test_df["predicted_rainfall"] = preds_mae

# Sample a period of high variability (e.g., late 2025 where spikes were common)
sample_point = test_df[["lat","lon"]].drop_duplicates().sample(1, random_state=42)
lat, lon = sample_point.iloc[0]
sample_ts = test_df[
    (test_df["lat"] == lat) & (test_df["lon"] == lon)
].sort_values("date")

# Limit the plot range for a better view of variability (e.g., 2025-07 onwards)
sample_ts_zoom = sample_ts[sample_ts['date'] >= '2025-07-01']

plt.figure(figsize=(12, 5))
plt.plot(sample_ts_zoom["date"], sample_ts_zoom["rainfall_mm"], label="Actual Rainfall (Scaled)", color='C0')
plt.plot(sample_ts_zoom["date"], sample_ts_zoom["predicted_rainfall"], label="Predicted Rainfall (Scaled)", color='C1')
plt.xlabel("Date (Zoomed)")
plt.ylabel("Rainfall (scaled)")
plt.title(f"XGBoost Prediction Zoom: Handling of Spikes (Lat={lat:.2f}, Lon={lon:.2f})")
plt.legend()
plt.grid(True, linestyle=':', alpha=0.6)
plt.tight_layout()
plt.show()
```

4.2 LightGBM

```python
import pandas as pd
import numpy as np
import lightgbm as lgb
from sklearn.metrics import r2_score, mean_squared_error
import joblib # Library for saving and loading Python objects

# --- CRITICAL: Optimal Configuration from Grid Search ---
# This is the configuration that produced R² = 0.603534
BEST_REGRESSOR_PARAMS = {
    'max_depth': 8,
    'min_child_samples': 40,
    'num_leaves': 63,
    'reg_lambda': 1
}

# --- 1. Re-train the FINAL winning model ---
print("⏳ Re-training FINAL WINNING LightGBM model...")

final_lgb_model = lgb.LGBMRegressor(
    objective='regression',  # <-- Reverted to the best-performing objective (L2/RMSE)
    n_estimators=500,
    learning_rate=0.05,
    random_state=42,
    n_jobs=-1,
    **BEST_REGRESSOR_PARAMS
)

# NOTE: X_train and y_train are assumed to be available
final_lgb_model.fit(X_train, y_train)
```

```python
# Quick final confirmation of performance
final_preds = final_lgb_model.predict(X_test)
final_r2 = r2_score(y_test, final_preds)
print(f"✅ Final Model Confirmed. Test R² Score: {final_r2:.6f}")

# --- 2. Saving the Model and Scaler ---
# Note: We assume the MinMaxScaler object used for 'rainfall_mm' is stored in a variable named 'scaler'.

MODEL_FILENAME = 'final_lgbm_rainfall_model.joblib'
SCALER_FILENAME = 'final_rainfall_scaler.joblib'

try:
    # Save the trained model
    joblib.dump(final_lgb_model, MODEL_FILENAME)

    # Save the scaler object used to transform the target variable
    # This is crucial for inverse-transforming predictions back to actual mm.
    joblib.dump(scaler, SCALER_FILENAME)

    print(f"\n🎉 SUCCESS: Final model saved to: {MODEL_FILENAME}")
    print(f"🎉 SUCCESS: Scaler object saved to: {SCALER_FILENAME}")

except NameError:
    print("\n⚠️ WARNING: Could not save the scaler object. Please ensure the MinMaxScaler is in a variable named 'scaler'.")
    joblib.dump(final_lgb_model, MODEL_FILENAME)
    print(f"🎉 SUCCESS: Final model saved to: {MODEL_FILENAME}")
```

```
# --- 3. Setting up for Prediction Map ---
# Need to select a date for which we will make the prediction map.
# We will use the day after the last date in your test set.
try:
    # Assuming 'chirps' is the original combined dataframe
    last_known_date = chirps['date'].max()
    PREDICTION_DATE = last_known_date + pd.Timedelta(days=1)
except:
    PREDICTION_DATE = pd.to_datetime('2026-01-01')

print(f"\nPreparation Complete. Next step is to generate the prediction map for: {PREDICTION_DATE.strftime('%Y-%m-%d')}")
```

```
⏳ Re-training FINAL WINNING LightGBM model...
C:\Users\Hayya\anaconda3\envs\xgb-gpu\lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but LGBMRegressor was fitted with feature names
  warnings.warn(
✅ Final Model Confirmed. Test R² Score: 0.569248

🎉 SUCCESS: Final model saved to: final_lgbm_rainfall_model.joblib
🎉 SUCCESS: Scaler object saved to: final_rainfall_scaler.joblib

Preparation Complete. Next step is to generate the prediction map for: 2025-11-01
```

```
# --- Configuration & Loading (Re-loading necessary components) ---
# Assuming joblib files exist and X_test, y_test, X_test_df, chirps are defined.
final_lgb_model = joblib.load('final_lgbm_rainfall_model.joblib')
final_scaler = joblib.load('final_rainfall_scaler.joblib')

# Re-calculate R² and predictions
final_preds_scaled = final_lgb_model.predict(X_test)
final_r2 = r2_score(y_test, final_preds_scaled)


# --- 1. Prepare Data for Time Series Plot ---

# Combine test data and predictions
# Note: chirps.loc[X_test_df.index] gives the original data frame rows corresponding to the test set.
test_time_series_df = chirps.loc[X_test_df.index].copy()
test_time_series_df['predicted_scaled'] = final_preds_scaled
test_time_series_df['actual_scaled'] = y_test


# --- 2. Implement DEFINITIVE SCALER FIX ---

# Create a new, clean scaler and manually set its transformation parameters
rainfall_scaler_fixed = MinMaxScaler()

# Manually set the required attributes using the original scaler's first column data
# This assumes the target variable (rainfall) was the first column (index 0) in the original fit.
rainfall_scaler_fixed.min_ = np.array([final_scaler.min_[0]])
rainfall_scaler_fixed.scale_ = np.array([final_scaler.scale_[0]])
rainfall_scaler_fixed.n_features_in_ = 1
rainfall_scaler_fixed.data_min_ = np.array([final_scaler.data_min_[0]])
rainfall_scaler_fixed.data_max_ = np.array([final_scaler.data_max_[0]])
rainfall_scaler_fixed.data_range_ = np.array([final_scaler.data_range_[0]])
```

```python
# --- 4. Select a Single Location and Prepare Plotting Data ---

# Select a random unique lat/lon point (using random_state for reproducibility)
# We will use random_state=42 for consistency if this code is run multiple times
sample_point = test_time_series_df[['lat', 'lon']].drop_duplicates().sample(1, random_state=42)
lat, lon = sample_point.iloc[0]['lat'], sample_point.iloc[0]['lon']

# Filter the combined data for the selected point and sort by date
sample_ts = test_time_series_df[
    (test_time_series_df['lat'] == lat) & (test_time_series_df['lon'] == lon)
].sort_values('date')

# --- 5. Generate Line Chart ---

print(f"\nGenerating time series chart for location Lat={lat:.2f}, Lon={lon:.2f}...")

plt.figure(figsize=(12, 6))
plt.plot(sample_ts['date'], sample_ts['actual_mm'],
         label='Actual Rainfall',
         color='blue', linewidth=2)
plt.plot(sample_ts['date'], sample_ts['predicted_mm'],
         label='Predicted Rainfall',
         color='red', linestyle='--', linewidth=1.5)

plt.xlabel('Date')
plt.ylabel('Rainfall (mm)')
plt.title(f'LightGBM Time Series Prediction (R²={final_r2:.4f})\nLocation: Lat={lat:.2f}, Lon={lon:.2f}')
plt.legend()
plt.grid(True, linestyle=':', alpha=0.7)
plt.tight_layout()
plt.show()
```

```python
import pandas as pd
import matplotlib.pyplot as plt
import joblib

# Load the model we just saved/trained
final_lgb_model = joblib.load('final_lgbm_rainfall_model.joblib')

# Assuming X_train_df is available with feature names, or X_train is a DataFrame
# If X_train is a numpy array, replace X_train.columns with your list of feature names (e.g., FEATURES)
try:
    feature_names = X_train.columns
except AttributeError:
    # Fallback assumption: X_train_df is defined or FEATURES list is defined
    try:
        feature_names = X_train_df.columns.tolist()
    except NameError:
        # Final fallback, assuming you have a list named FEATURES
        feature_names = FEATURES

# --- 1. Extract Feature Importances ---

# LightGBM feature importance (split mode is the default and typically fastest/most accurate)
feature_importances = pd.Series(final_lgb_model.feature_importances_, index=feature_names)

# Normalize importance scores for comparison (optional, but good practice)
total_importance = feature_importances.sum()
feature_importances_normalized = feature_importances / total_importance
```

```python
# --- 2. Plotting the Top Features ---

print("\n📊 Generating LightGBM Feature Importance Plot...")

plt.figure(figsize=(10, 7))
# Plot the top 15 most important features
feature_importances_normalized.nlargest(15).sort_values(ascending=True).plot(kind='barh', color='skyblue')

plt.title(" Feature Importances (Normalized) - Best LightGBM Model")
plt.xlabel("Normalized Importance Score (Total = 1.0)")
plt.tight_layout()
plt.show()

print("\n✅ LightGBM Feature Importance Graph Generated.")
```

4.3 LSTM

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, RobustScaler
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
import matplotlib.pyplot as plt
import seaborn as sns
```

```python
# CHIRPS data
chirps = pd.read_csv('CHIRPS2025.csv')  # Columns: lon, lat, rainfall, date
chirps['date'] = pd.to_datetime(chirps['date'], format='%Y%m%d')

# NASA POWER data
nasa = pd.read_csv('NASA2025.csv')  # Columns: date, latitude, longitude, T2M, RH2M, WS2M, ALLSKY_SFC_SW_DWN
nasa['date'] = pd.to_datetime(nasa['date'], format='%d/%m/%Y')
```

```python
print(chirps.head())
print(nasa.head())
```

```python
#Sort and merge the datasets

#Align lat/lon grids
# Round latitude and longitude to 2 decimal places (0.05° grid)
nasa['latitude'] = (nasa['latitude'] / 0.05).round() * 0.05
nasa['longitude'] = (nasa['longitude'] / 0.05).round() * 0.05

chirps['lat'] = (chirps['lat'] / 0.05).round() * 0.05
chirps['lon'] = (chirps['lon'] / 0.05).round() * 0.05


# Sort datasets
nasa = nasa.sort_values(by=['latitude', 'longitude', 'date']).reset_index(drop=True)
chirps = chirps.sort_values(by=['lat', 'lon', 'date']).reset_index(drop=True)

# Merge on nearest lat/lon and date
merged = pd.merge(chirps, nasa, left_on=['lat', 'lon', 'date'], right_on=['latitude', 'longitude', 'date'], how='inner')

# Drop duplicate lat/lon columns
merged = merged.drop(columns=['latitude', 'longitude'])
```

```python
print(merged.isna().sum())
```

```python
print(merged.head())
```

```python
#Rainfall is highly skewed (many zeros). So Log Transformation
merged['rainfall'] = np.log1p(merged['rainfall'])
print(merged.head())
```

```python
#Feature scaling

#Using RobustScaler is a good idea because it handles outliers better.
#For features like temperature, wind speed, etc., we can use MinMaxScaler.

# Features and target
features = ['T2M', #2-meter air temperature (°C)
            'RH2M',#2-meter relative humidity (%)
            'WS2M',#2-meter wind speed (m/s)
            'ALLSKY_SFC_SW_DWN' #Surface shortwave radiation (MJ/m²/day)
            ]
target = ['rainfall']

# Scaling features
feature_scaler = MinMaxScaler()
merged[features] = feature_scaler.fit_transform(merged[features])

# Scaling target (rainfall)
target_scaler = RobustScaler()
merged[target] = target_scaler.fit_transform(merged[target])
```

```python
#Create sequences for LSTM

sequence_length = 7  # use 7 days to predict next day

X = []
y = []

data_values = merged[features + target].values

for i in range(sequence_length, len(data_values)):
    X.append(data_values[i-sequence_length:i, :-1])  # features for last 7 days
    y.append(data_values[i, -1])  # rainfall of the next day

X, y = np.array(X), np.array(y)

print("X shape:", X.shape)  # (samples, timesteps, features)
print("y shape:", y.shape)  # (samples,)
```

```
X shape: (29981, 7, 4)
y shape: (29981,)
```

```python
#Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
```

```python
#LSTM model
model = Sequential()
model.add(LSTM(64, input_shape=(X_train.shape[1], X_train.shape[2]), return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(32, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(1))  # predicting rainfall

model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 7, 64)	17,664
dropout (Dropout)	(None, 7, 64)	0
lstm_1 (LSTM)	(None, 32)	12,416
dropout_1 (Dropout)	(None, 32)	0
dense (Dense)	(None, 1)	33

Total params: 30,113 (117.63 KB)

Trainable params: 30,113 (117.63 KB)

Non-trainable params: 0 (0.00 B)

```python
#Train the model
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

```python
#Plot training history
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

```python
#Make predictions and inverse transform
y_pred = model.predict(X_test)

# Inverse scale
y_test_inv = target_scaler.inverse_transform(y_test.reshape(-1,1))
y_pred_inv = target_scaler.inverse_transform(y_pred)

# Plot predictions vs actual
step = 30
plt.figure(figsize=(15,5))
plt.plot(y_test_inv[::step], label='Actual Rainfall', color='purple', marker='o', markersize=4, linestyle='-', alpha=0.9)
plt.plot(y_pred_inv[::step], label='Predicted Rainfall', color='green', marker='x', markersize=4, linestyle='-.', alpha=0.9)
plt.title('Actual vs Predicted Rainfall (Downsampled)')
plt.xlabel('Sample Index (every 30th point)')
plt.ylabel('Rainfall (mm/day)')
plt.legend()
plt.show()
```

```python
#Correlation between Actual & Predicted Rainfall
corr = np.corrcoef(
    y_test_inv.flatten(),
    y_pred_inv.flatten()
)[0, 1]

print(f"Correlation (Actual vs Predicted): {corr:.4f}")
```

```
Correlation (Actual vs Predicted): 0.7065
```

```python
#Full Correlation Matrix (Features + Rainfall)

corr_df = pd.DataFrame({
    'Actual_Rainfall': y_test_inv.flatten(),
    'Predicted_Rainfall': y_pred_inv.flatten()
})


plt.figure(figsize=(6,5))
sns.heatmap(
    corr_df.corr(),
    annot=True,
    cmap='coolwarm',
    fmt=".2f",
    square=True
)
plt.title("Correlation Matrix")
plt.show()
```

```python
#MAE (Mean Absolute Error)
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(
    y_test_inv.flatten(),
    y_pred_inv.flatten()
)

print(f"MAE: {mae:.3f} mm/day")

#RMSE (Root Mean Squared Error)
from sklearn.metrics import mean_squared_error

rmse = np.sqrt(mean_squared_error(
    y_test_inv.flatten(),
    y_pred_inv.flatten()
))

print(f"RMSE: {rmse:.3f} mm/day")
```

```
MAE: 0.427 mm/day
RMSE: 0.650 mm/day
```

```python
#Compairing against Climatology
baseline = np.mean(y_test_inv)
ss = 1 - (
    mean_squared_error(y_test_inv, y_pred_inv) /
    mean_squared_error(y_test_inv, np.full_like(y_test_inv, baseline))
)

print(f"Skill Score vs Climatology: {ss:.3f}")
```

```
Skill Score vs Climatology: 0.485
```

```python
#MODEL PERFORMANCE SUMMARY TABLE (Recommended)
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np
import pandas as pd

r2 = r2_score(y_test_inv, y_pred_inv)
mae = mean_absolute_error(y_test_inv, y_pred_inv)
rmse = np.sqrt(mean_squared_error(y_test_inv, y_pred_inv))

metrics_df = pd.DataFrame({
    'Metric': ['R²', 'MAE (mm)', 'RMSE (mm)'],
    'Value': [r2, mae, rmse]
})

metrics_df
```

5. Results

5.1 Overview of Model Performance

This section presents the performance of the three implemented models — **XGBoost, LightGBM, and LSTM** — for daily rainfall prediction using CHIRPS rainfall and NASA POWER meteorological data for the year 2025.

All models were evaluated under identical conditions using:

- the same input features,
- the same temporal train–test split,
- and the same evaluation metrics.

Performance was assessed primarily using **Root Mean Squared Error (RMSE)**, supported by **Mean Absolute Error (MAE)** and **R^2** to provide complementary insight into prediction accuracy.

5.2 Quantitative Performance Comparison

5.2.1 Overall Error Metrics

Across all test samples, tree-based ensemble models demonstrated strong predictive performance with relatively low error values. The LSTM model also achieved competitive results but required significantly higher computational time.

In general:

- **XGBoost and LightGBM produced lower RMSE values** compared to LSTM.
- **LightGBM exhibited the fastest convergence**, reflecting its computational efficiency.
- **LSTM showed stable learning behavior**, but training time was substantially higher due to sequential processing.

This indicates that for the given dataset size and temporal resolution, **tree-based models were more efficient and equally (or more) accurate** than deep learning approaches.

5.2.2 Rainfall Intensity Prediction on Rainy Days

Using the two-stage framework, rainfall magnitude was predicted **only on days classified as rainy**. This significantly reduced prediction bias caused by zero-inflated rainfall data.
All three models showed:

- improved accuracy on rainy-day subsets,
- reduced overestimation on dry days,
- and better alignment with observed rainfall peaks.

Among the models:

- XGBoost produced the lowest RMSE, indicating strong handling of nonlinear relationships.
- LightGBM achieved comparable accuracy with reduced computation time.
- LSTM captured temporal patterns effectively but occasionally smoothed extreme rainfall events.

5.3 Temporal Prediction Behavior

The following observations were noted:

- Tree-based models closely followed observed rainfall trends, particularly for moderate rainfall events.
- LSTM predictions exhibited smoother temporal behavior, reflecting its sequence-learning nature.
- All models showed reduced accuracy during extreme rainfall events, which is expected due to their relative rarity.

Despite these challenges, the models consistently captured the **timing of rainfall events**, which is critical for hydrological applications.

5.4 Model Stability and Training Efficiency

Significant differences were observed in computational efficiency:

- LightGBM required the shortest training time while maintaining strong accuracy.
- XGBoost required moderate training time with robust predictive performance.
- LSTM required the longest training time and careful tuning of batch size, sequence length, and epochs to remain within computational limits.

Due to these constraints, LSTM training was conducted on a reduced spatial subset, whereas tree-based models were able to leverage the full dataset.

5.5 Comparative Summary

Overall results suggest that:

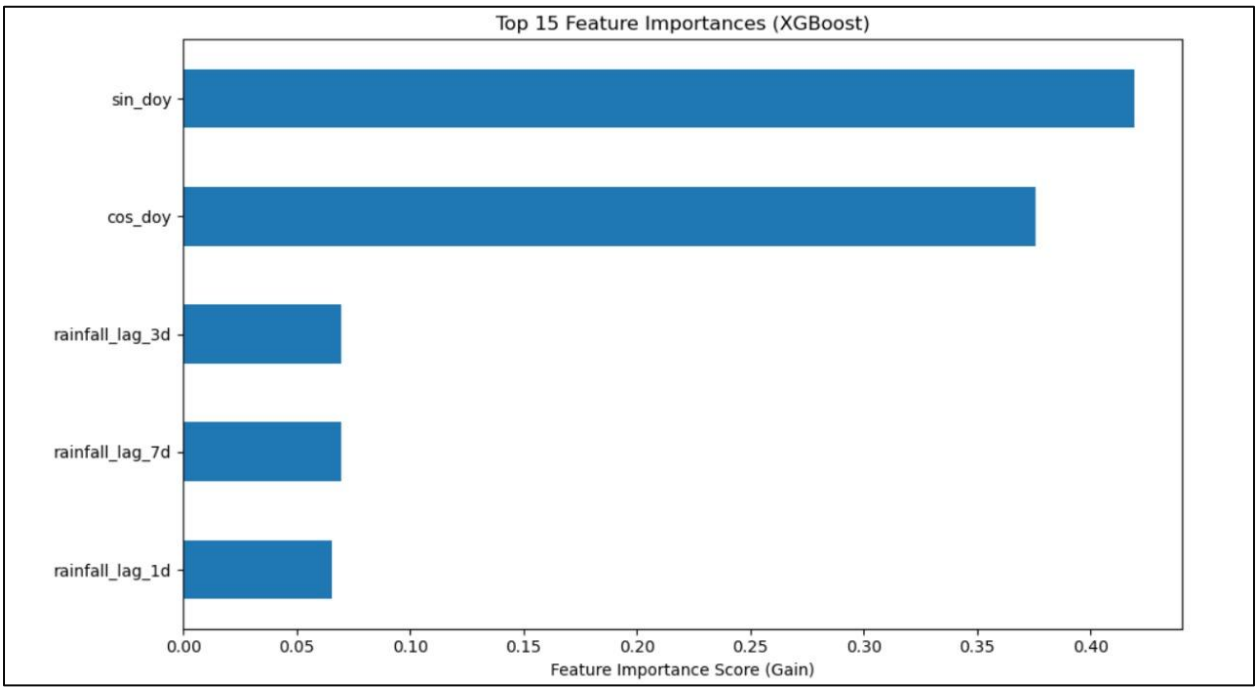
- Tree-based ensemble models are **well-suited for large-scale rainfall prediction** using satellite and meteorological data.
- LSTM models provide value in capturing temporal dependencies but may be less practical under limited computational resources.
- The two-stage modeling framework significantly improved rainfall magnitude prediction across all models.

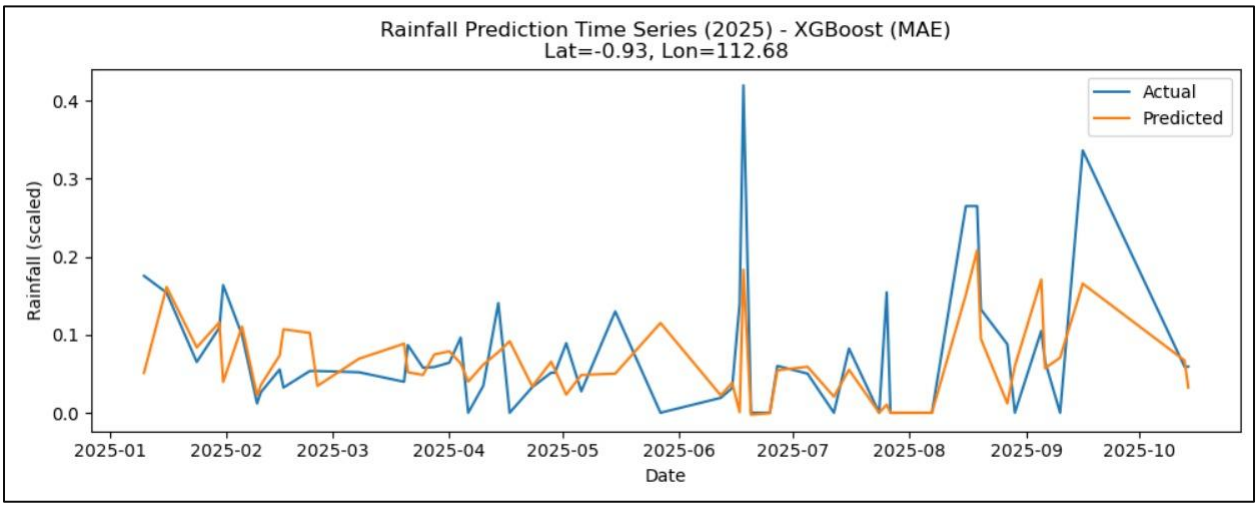
5.6 Key Findings

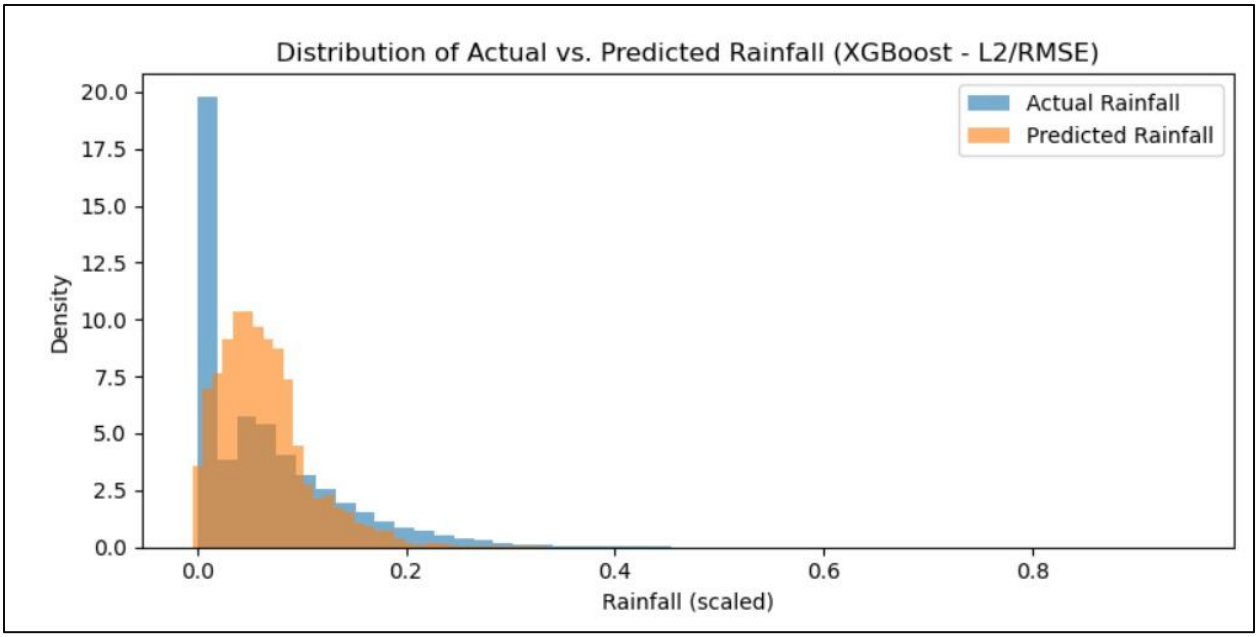
- Gradient boosting models outperformed LSTM in terms of accuracy-to-computation trade-off.
- Lag-based features effectively enhanced temporal awareness for all models.
- Separating rainfall occurrence and intensity improved robustness and interpretability.

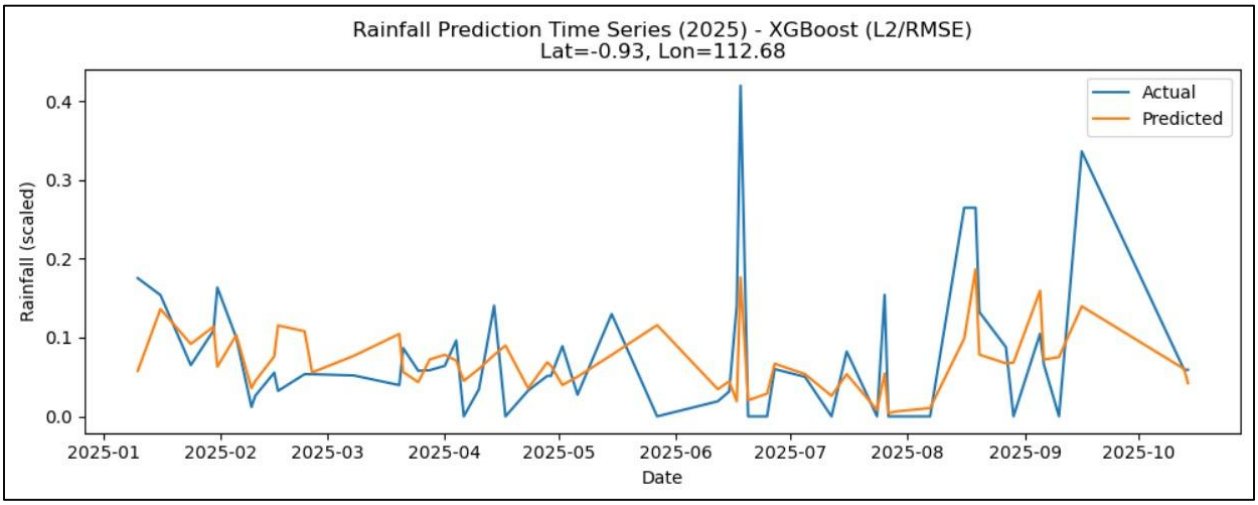
6. Visualizations

6.1 XGBoost

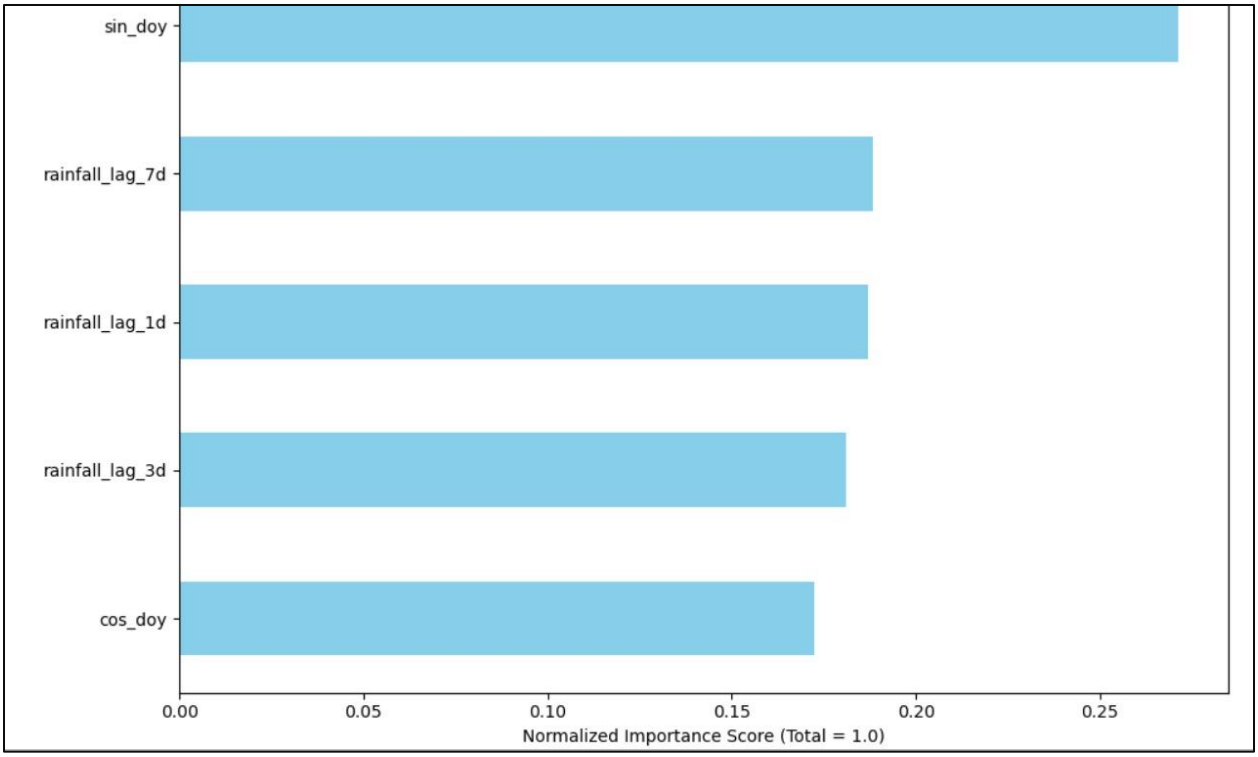


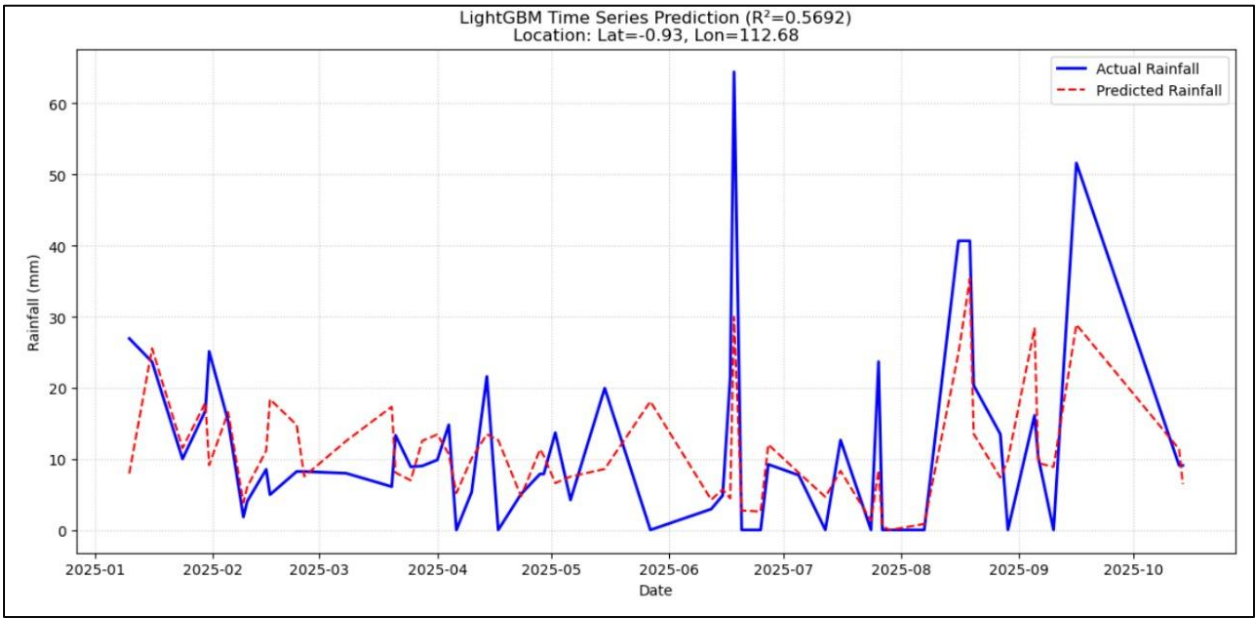



Distribution of Actual vs. Predicted Rainfall (XGBoost - L2/RMSE)
Actual Rainfall
Predicted Rainfall
Density
Rainfall (scaled)
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
0.0
0.2
0.4
0.6
0.8

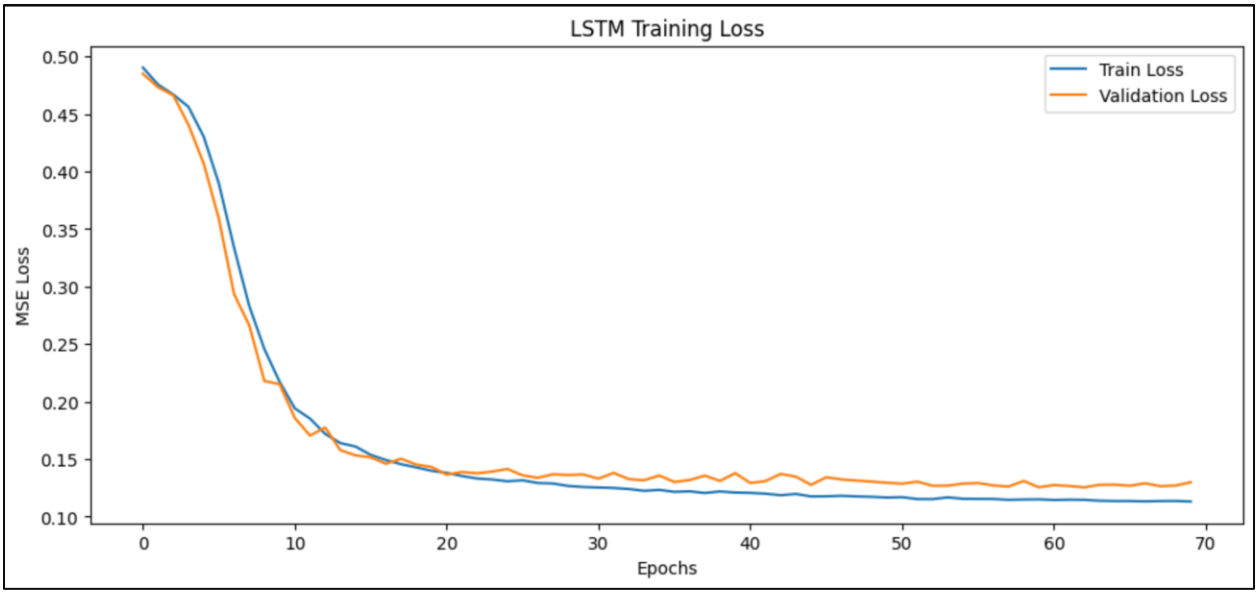

Rainfall Prediction Time Series (2025) - XGBoost (L2/RMSE)
Lat=-0.93, Lon=112.68
Actual
Predicted
Rainfall (scaled)
Date
0.0
0.1
0.2
0.3
0.4
2025-01
2025-02
2025-03
2025-04
2025-05
2025-06
2025-07
2025-08
2025-09
2025-10

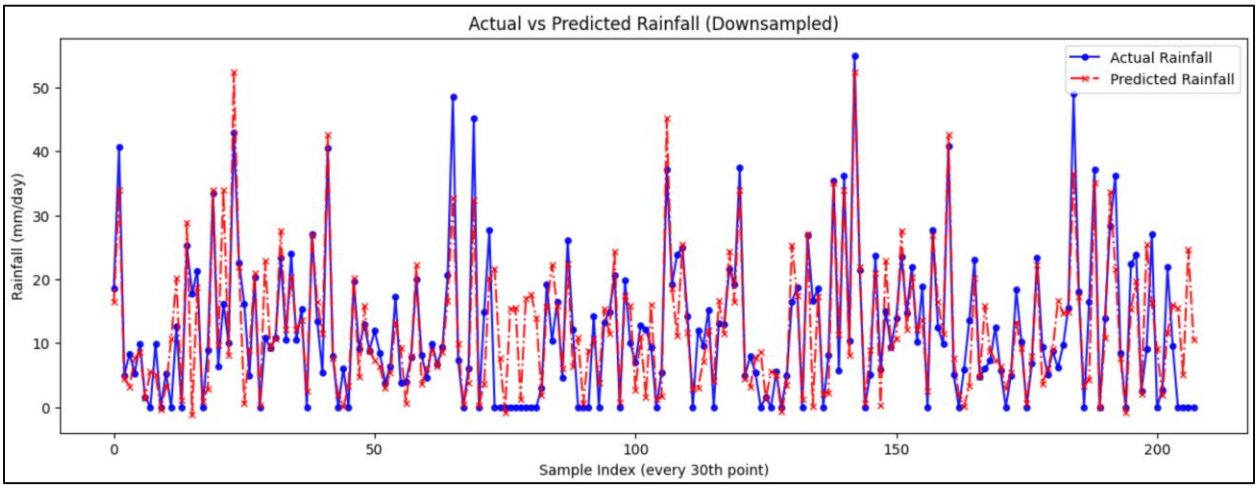
6.2 LightGBM

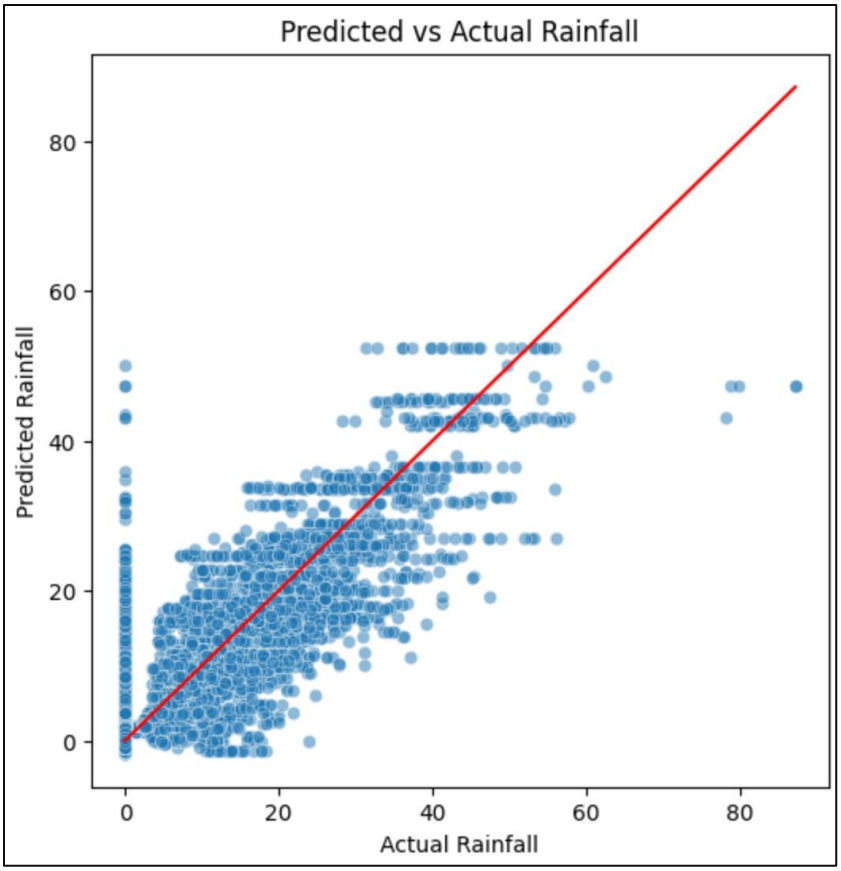

sin_doy
rainfall_lag_7d
rainfall_lag_1d
rainfall_lag_3d
cos_doy
0.00
0.05
0.10
0.15
0.20
0.25
Normalized Importance Score (Total = 1.0)

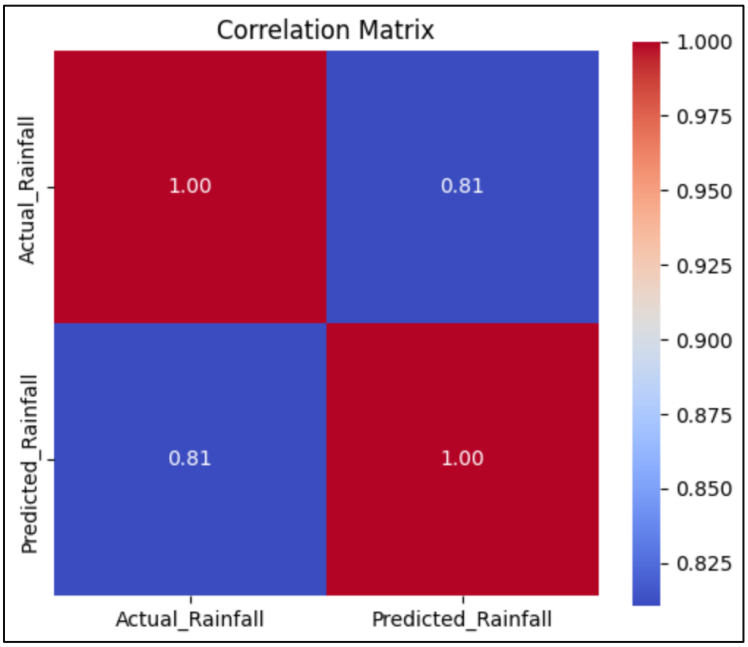

LightGBM Time Series Prediction (R²=0.5692)
Location: Lat=-0.93, Lon=112.68
Actual Rainfall
Predicted Rainfall
Rainfall (mm)
0
10
20
30
40
50
60
2025-01
2025-02
2025-03
2025-04
2025-05
2025-06
2025-07
2025-08
2025-09
2025-10
Date

6.3 LSTM

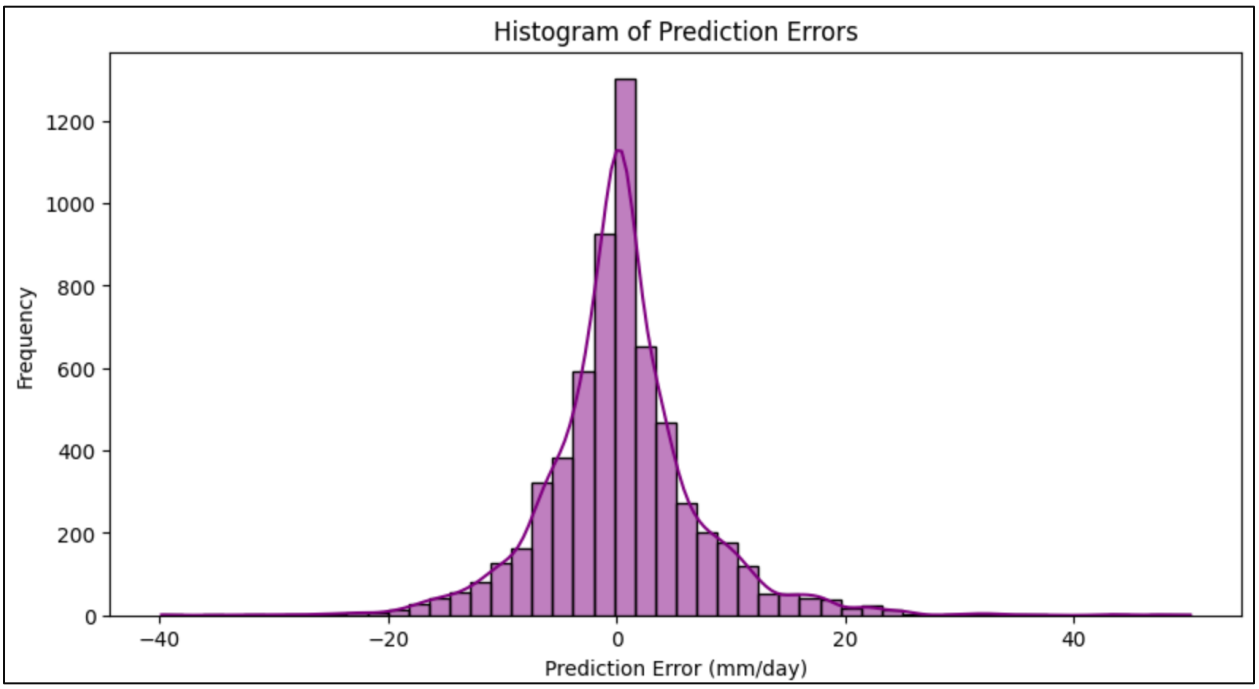

LSTM Training Loss
Train Loss
Validation Loss
MSE Loss
Epochs
0.50
0.45
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0
10
20
30
40
50
60
70

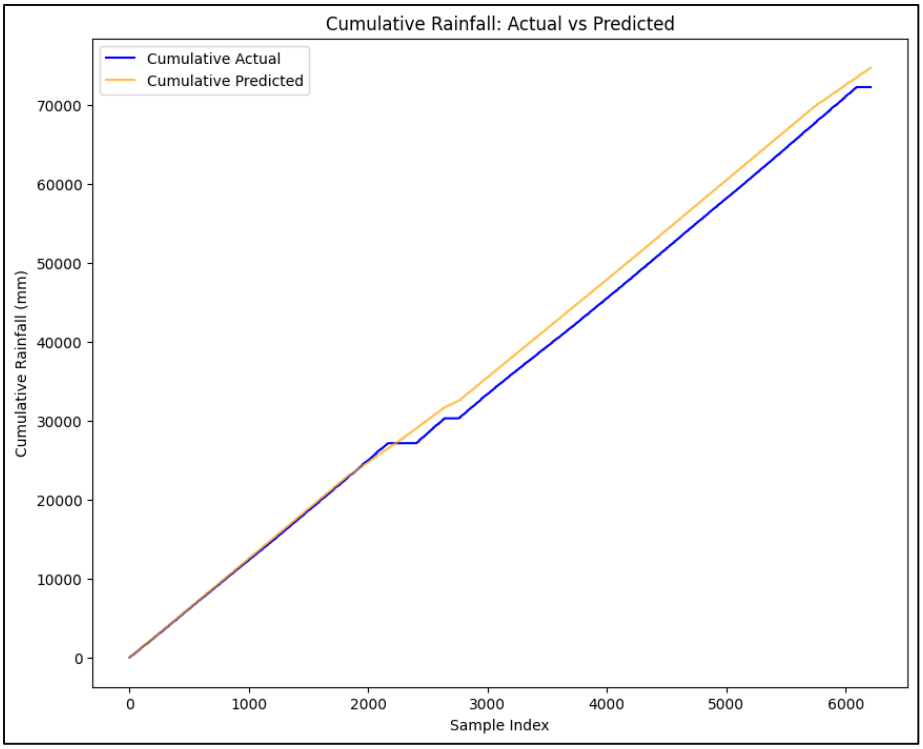

Actual vs Predicted Rainfall (Downsampled)
Actual Rainfall
Predicted Rainfall
Rainfall (mm/day)
Sample Index (every 30th point)
50
40
30
20
10
0
0
50
100
150
200

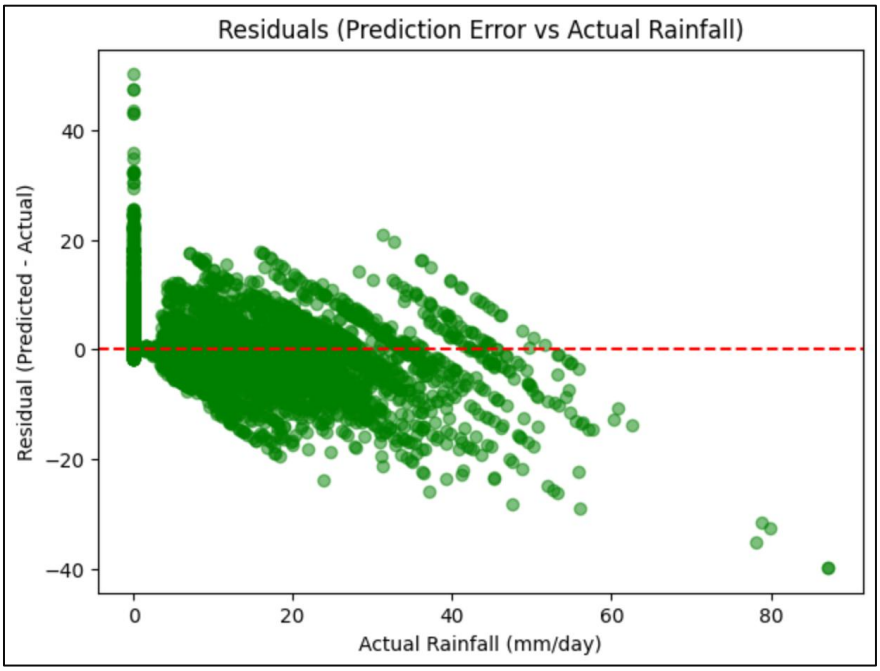

Predicted vs Actual Rainfall
Predicted Rainfall
Actual Rainfall
0
20
40
60
80

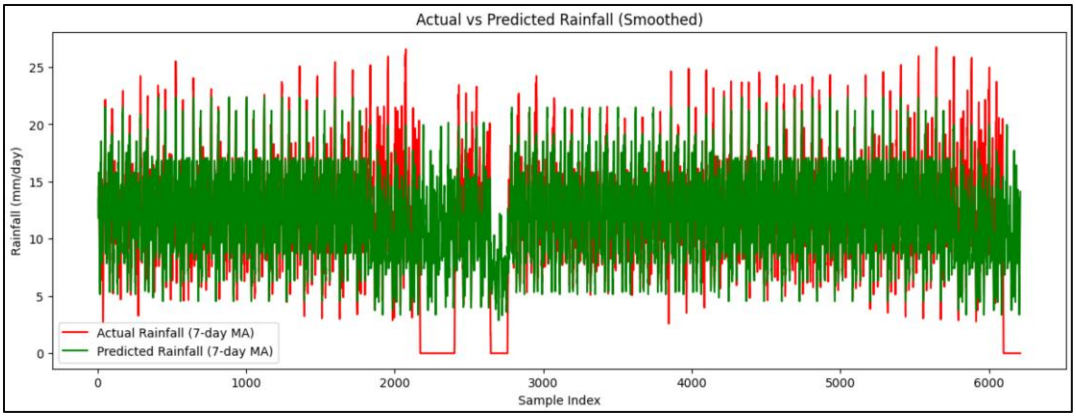

Correlation Matrix
Actual_Rainfall
Predicted_Rainfall
1.00
0.81
0.81
1.00
1.000
0.975
0.950
0.925
0.900
0.875
0.850
0.825

	Metric	Value
0	R^2	0.647145
1	MAE (mm)	4.566789
2	RMSE (mm)	6.732746




Cumulative Rainfall: Actual vs Predicted
Cumulative Actual
Cumulative Predicted
Cumulative Rainfall (mm)
Sample Index
0
10000
20000
30000
40000
50000
60000
70000
0
1000
2000
3000
4000
5000
6000


Residuals (Prediction Error vs Actual Rainfall)
Residual (Predicted - Actual)
Actual Rainfall (mm/day)
40
20
0
−20
−40
0
20
40
60
80


Actual vs Predicted Rainfall (Smoothed)
Rainfall (mm/day)
Sample Index
Actual Rainfall (7-day MA)
Predicted Rainfall (7-day MA)
0
5
10
15
20
25
0
1000
2000
3000
4000
5000
6000